```
    }
}
```

❶ The handler implements AuthorizationHandler<T>.

❷ You must override the abstract HandleRequirementAsync method.

❸ The context contains details such as the ClaimsPrincipal user object.

❹ The requirement instance to handle

❺ Checks whether the user has the Frequent-FlyerClass claim with the Gold value

❻ If the user had the necessary claim, marks the requirement as satisfied by calling Succeed

❼ If the requirement wasn't satisfied, does nothing

This handler is functionally equivalent to the simple `RequireClaim()` handler you saw at the start of section 24.4, but using the requirement and handler approach instead.

When a request is routed to the AirportLounge.cshtml Razor Page, the authorization middleware sees the `[Authorize]` attribute on the endpoint with the `"CanAccessLounge"` policy. It loops through all the requirements in the policy and all the handlers for each requirement, calling the `HandleRequirementAsync` method for each.

The authorization middleware passes the current `AuthorizationHandlerContext` and the requirement to be checked to each handler. The current `ClaimsPrincipal` being authorized is exposed on the context as the `User` property. In listing 24.8, `FrequentFlyerHandler` uses the context to check for a claim called `FrequentFlyerClass` with the `Gold` value, and if it exists, indicates that the user is allowed to enter the airline lounge by calling `Succeed()`.

NOTE Handlers mark a requirement as being satisfied by calling `context .Succeed()` and passing the requirement as an argument.